

Programming Foundations, Compiler and Data Structures — Detailed Lecture Notes

JKSSB Computer Awareness

October 8, 2026

51.1 Programming languages and their levels

A computer can act only on instructions given in its own **machine language**, a pattern of binary 0s and 1s. Writing whole programs in that form is slow, error-prone, and tied to one processor. Programmers therefore write in languages that are easier for humans, and a translator converts the resulting program into machine language. Languages are placed on a **level** scale: low-level languages (machine and assembly) are close to the hardware, while high-level languages are closer to human language. The lower the level, the more closely the code reflects the actual machine instructions.

Definition (Level of a programming language). *How close a language is to the underlying machine. Machine and assembly languages are low-level; C, C++, Java, and Python are high-level.*

Language	Generation	Main feature
Machine language	First	Binary 0s and 1s; only language the CPU runs directly
Assembly language	Second	Short mnemonics such as ADD and MOV; machine-dependent
High-level language	Third	English-like and mathematical; machine-independent

51.2 Machine language

Machine language is the only language a processor executes directly. It is written entirely in binary, and it is *machine-dependent*: each processor family has its own instruction set, so a program written for one machine does not run on another. Machine language needs no translation, so it is fast to run, but it is very difficult to write, read, and debug by hand. It is also called the *first-generation language*.

51.3 Assembly language

Assembly language replaces binary instruction codes with short *mnemonics* such as ADD, SUB, and MOV. It is still *machine-dependent*, because each processor family has its own assembly language, and it maps almost one-to-one onto machine instructions. Assembly code cannot run directly; it must be translated into machine language by an **assembler**. Assembly language is called the *second-generation language* and is a low-level language.

A common exam trap is to call assembly language a high-level language. It is not: assembly language is a low-level, second-generation language because its statements correspond closely to machine instructions. High-level languages, by contrast, are machine-independent and use English-like statements.

51.4 High-level languages

High-level languages use English-like words and mathematical notation. Examples are C, C++, Java, Python, COBOL, and FORTRAN. They are *machine-independent*: after translation, the same source program can run on different machines. They are easier to learn, write, read, and maintain than low-level languages, but they must be translated by a **compiler** or an **interpreter** before the CPU can run them.

51.5 Generations of programming languages

Languages are also grouped into generations, which run parallel to the level scale above. The first three generations match the three levels exactly.

Generation	Example	Main feature
First (1GL)	Machine language	Binary 0s and 1s; run directly
Second (2GL)	Assembly language	Mnemonics; needs an assembler
Third (3GL)	C, C++, Java	High-level; needs a compiler or interpreter
Fourth (4GL)	SQL	Non-procedural; states what, not how
Fifth (5GL)	Logic / AI languages	Problem-solving and constraints

For this exam, the high-yield rows are the first three: machine language is first generation, assembly language is second generation, and a high-level language such as C or Java is third generation.

51.6 Language translators: assembler, compiler, interpreter

A **translator** is a program that converts source code written in one language into another form, usually machine language. There are three common translators, each matched to a different input language and method.

Definition (Assembler). *A translator that converts an assembly-language program into machine code.*

Definition (Compiler). *A translator that converts a high-level program into machine code (object code) as a whole, translating the entire program before it runs.*

Definition (Interpreter). *A translator that converts and executes a high-level program line by line, without producing a separate object file.*

Point	Compiler	Interpreter
Input	Whole program at once	One statement at a time
Output	Object code file	No separate object file
Execution speed	Faster	Slower
Errors	Reported after full translation	Reported line by line
Debugging	Harder	Easier
Typical examples	C, C++	Python, JavaScript

The compiler translates the whole program in one pass and produces object code, so execution is fast, but a single error can delay the whole build and debugging is harder. The interpreter takes one statement at a time, translates it, and executes it immediately; execution is slower, but errors are shown line by line, which makes debugging easier.

51.7 Source code, object code, and machine code

These three terms describe a program at different stages of translation.

Definition (Source code). *The program as written by the programmer in a high-level or assembly language; it is human-readable text.*

Definition (Object code). *The machine-readable, relocatable code produced by a compiler or assembler from source code; it is not yet linked into a final runnable program.*

Definition (Machine code). *The final binary instructions, in the processor's own language, that the CPU executes directly.*

The path is: the programmer writes **source code**; the translator converts it into **object code**; after linking, the result is **machine code** that the processor runs. Source code is human-readable; object code and machine code are machine-readable.

A common exam trap is to ask which of these is written by the programmer. The answer is **source code**. Object code and machine code are produced by the translator, not written by hand, and they are stored as binary patterns rather than readable text.

How a program is translated

The translation of a high-level program follows a fixed sequence:

1. The programmer writes the program as **source code**.
2. A **compiler** translates the whole source file and produces an **object file**.
3. A **linker** joins the object file with any library code needed.
4. The result is a **machine-code** executable that the CPU runs.

An interpreter shortens this path: it takes each statement of the source program, translates it, and executes it immediately, so no separate object file is produced.

51.8 Algorithm

An **algorithm** is a finite, step-by-step procedure for solving a problem. It has a clear start and end, and every step is unambiguous so that anyone following it reaches the same result. An algorithm

is language-independent: it can be written in plain English or in pseudocode, without committing to a particular programming language. A program is the algorithm expressed in a programming language.

Definition (Algorithm). *A finite, ordered set of unambiguous steps that solves a problem, with a defined start and end.*

Example 51.8.1. *Adding two numbers is a small algorithm: (1) start, (2) read values a and b , (3) compute $sum = a + b$, (4) display sum, (5) stop. The same steps can later be written in any programming language.*

51.9 Flowchart

A **flowchart** is a pictorial, diagrammatic representation of an algorithm. It uses standard symbols joined by arrows to show the order in which steps are carried out, which makes the flow of control easy to see at a glance.

Symbol	Meaning
Oval (terminator)	Start and end of the process
Parallelogram	Input or output
Rectangle	Process (a computation or action)
Diamond (rhombus)	Decision (yes/no branch)
Arrow	Flow line showing direction

A flowchart for the add-two-numbers algorithm uses these symbols in order: an oval for start, a parallelogram to read the two inputs, a rectangle to compute the sum, a parallelogram to display the result, and an oval for stop. When a step involves a choice, a diamond is used, and two arrows leave it, one for each outcome.

51.10 Debugging and types of errors

Debugging is the process of finding and correcting errors, called *bugs*, in a program. Three kinds of error are commonly distinguished.

Definition (Syntax error). *A violation of the rules of the language, such as a missing semicolon or a misspelled keyword. It is reported by the compiler.*

Definition (Logical error). *An error in the logic of the program: it compiles and runs but produces a wrong result. It is the hardest kind to find.*

Definition (Runtime error). *An error that occurs while the program is running, such as dividing by zero or reading a file that does not exist.*

A compiler catches syntax errors but not logical errors. A program with a logical error compiles and runs, and only the output reveals that something is wrong.

51.11 Object-oriented programming basics

OOP stands for **Object-Oriented Programming**. It organises a program around objects rather than around a sequence of actions. Two central terms are the class and the object.

Definition (Class and object). *A class is a blueprint or template that defines the data and methods; an object is a single instance created from that class.*

Four features define the object-oriented approach.

- **Encapsulation:** binding the data and the methods that operate on it into one unit, with the data hidden from outside access.
- **Inheritance:** a new class acquires the properties and methods of an existing class.
- **Polymorphism:** “one name, many forms” — the same operation behaves differently for different objects.
- **Abstraction:** hiding the implementation details and showing only the essential features.

As an illustration, *Student* can be a class that defines the data (roll number, name, marks) and the methods (enrol, show result). A particular student such as one with roll number 12 is then an *object* of that class. The class describes the shape; each object is one concrete example of it.

51.12 Stack and LIFO

A **stack** is a linear data structure in which insertion and deletion take place at only one end, called the *top*. It follows the order **LIFO**: Last In, First Out. The last element pushed onto the stack is the first element popped from it.

Definition (Stack (LIFO)). *A linear data structure that works on the Last In, First Out rule, with insertion and deletion performed at the top.*

Insertion into a stack is called **PUSH** and deletion is called **POP**. The other end of the stack is the *bottom*, and no insertion or deletion happens there.

Example 51.12.1. *A stack of files on a desk behaves in exactly this way. The file placed on top last is the first one picked up next. That is the LIFO rule in everyday form.*

51.13 Basic data structures

A data structure is a way of organising data so that it can be used efficiently. The common structures tested at this level are the array, stack, queue, linked list, and tree.

Structure	Defining property
Array	Fixed-size elements of the same type, stored in contiguous memory and accessed by an index
Stack	Last In, First Out (LIFO); push and pop take place at the top
Queue	First In, First Out (FIFO); insertion at the rear, deletion at the front
Linked list	Nodes that hold data and a pointer to the next node; not contiguous and can grow dynamically
Tree	A hierarchical structure with a root node and child nodes

To identify a structure in an exam item, look for its defining rule: same type, fixed size, and index access means an **array**; Last In, First Out means a **stack**; First In, First Out (like people in a queue) means a **queue**; data-plus-pointer nodes that grow dynamically mean a **linked list**; and a root with branches means a **tree**.

The two most commonly confused structures are the stack and the queue. They differ only in the order in which elements leave the structure.

Point	Stack	Queue
Order	LIFO (Last In, First Out)	FIFO (First In, First Out)
Insertion	Push, at the top	Enqueue, at the rear
Deletion	Pop, at the top	Dequeue, at the front
Both ends used?	No, one end only	Yes, front and rear

51.14 Exam retrieval and common confusions

The following distinctions prevent most errors on this topic.

1. A **compiler** translates the **whole program at once**; an **interpreter** works **line by line**.
2. An **assembler** translates **assembly language**; a **compiler** and an **interpreter** translate **high-level** languages.
3. **Source code** is human-readable; **object code** and **machine code** are machine-readable.
4. An **algorithm** is a step-by-step procedure; a **flowchart** is its pictorial representation.
5. A **stack** is **LIFO**; a **queue** is **FIFO**. Do not swap them.
6. A **logical error** is not caught by the compiler; the program runs but the result is wrong.
7. **OOP** stands for Object-Oriented Programming; a class is a blueprint and an object is an instance of it.

Example 51.14.1. *An item asks which translator produces a separate object file before execution. Trace the definitions: the compiler translates the whole high-level program and produces object code, so the answer is the compiler. If the stem instead says “translates and executes line by line”, the answer is the interpreter, and if it says “translates assembly language”, the answer is the assembler.*

Exercises

1. Distinguish machine language, assembly language, and a high-level language by level and by dependence on the machine.
2. State the difference between an assembler, a compiler, and an interpreter.
3. Differentiate source code, object code, and machine code.
4. Define an algorithm and explain how a flowchart represents one.
5. Distinguish a syntax error, a logical error, and a runtime error.
6. Explain LIFO and name the two operations of a stack.
7. Identify the data structure: (a) elements of the same type in contiguous memory, (b) Last In First Out, (c) a root with child nodes.

Solutions

1. Machine language is the first-generation, binary, machine-dependent language the CPU runs directly. Assembly language is the second generation, uses mnemonics, and is also machine-dependent. A high-level language is third generation, uses English-like statements, and is machine-independent.
2. An assembler translates assembly language into machine code. A compiler translates a high-level program as a whole and produces object code. An interpreter translates and executes a high-level program line by line.
3. Source code is the program written by the programmer in a high-level or assembly language and is human-readable. Object code is the machine-readable code produced by the translator and is not yet linked. Machine code is the final binary code the CPU executes directly.
4. An algorithm is a finite, step-by-step, unambiguous procedure to solve a problem. A flowchart represents it pictorially using standard symbols (oval, parallelogram, rectangle, diamond, arrow) joined by flow lines.
5. A syntax error breaks the rules of the language and is reported by the compiler. A logical error lets the program run but gives a wrong result. A runtime error occurs while the program is running.
6. LIFO means Last In, First Out: the last element pushed is the first element popped. The two operations are push (insertion) and pop (deletion), both performed at the top.
7. (a) Array, (b) Stack, (c) Tree.